

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут телекомунікацій, радіоелектроніки та електронної техніки



Лабораторна робота
З дисципліни
“Об'єктно-орієнтоване програмування, частина 2”

Виконав: ст.гр IX-22

Горячий Ростислав

Прийняв:

Гордійчук-Бублівська О.В

Львів – 2026

Мета роботи.

Засвоїти основи роботи з бібліотекою Pandas, навчитися працювати з табличними даними, виконувати фільтрацію, сортування, групування та застосовувати функції для аналітики.

1. Імпорт та первинний аналіз даних

Завантажити CSV-файл у DataFrame.

Вивести перші 5 та останні 5 рядків таблиці.

Визначити кількість рядків і стовпців.

Обчислити обсяг пам'яті, який займає датасет.

```
et.py > ...
1  import pandas as pd
2  import os
3
4  file_path = "data.csv"
5  |
6  if not os.path.exists(file_path):
7      print("Файл не знайдено:", file_path)
8
9  elif os.path.getsize(file_path) == 0:
10     print("Файл порожній")
11
12  else:
13      try:
14          df = pd.read_csv(file_path)
15
16          print("Перші 5 рядків:")
17          print(df.head())
18
19          print("\nОстанні 5 рядків:")
20          print(df.tail())
21
22          rows, cols = df.shape
23          print("\nКількість рядків:", rows)
24          print("Кількість стовпців:", cols)
25
26          memory = df.memory_usage(deep=True).sum()
27          print("\nОбсяг пам'яті:", memory, "bytes")
28
29      except Exception as e:
30          print("Помилка при читанні файлу:", e)
```

```
PS C:\Users\w1olf\Downloads\dw> & C:/Users/w1olf/AppData/Local/Py
```

Перші 5 рядків:

	Name	Age	City
0	yura	20	Lviv
1	antyk	19	wroclaw
2	komuto herovato	22	Kyoto
3	mina	20	munich

Останні 5 рядків:

	Name	Age	City
0	yura	20	Lviv
1	antyk	19	wroclaw
2	komuto herovato	22	Kyoto
3	mina	20	munich

Кількість рядків: 4

Кількість стовпців: 3

Обсяг пам'яті: 607 bytes

```
PS C:\Users\w1olf\Downloads\dw>
```

```
et.py > ...
```

```
1 import pandas as pd
2
3 file_path = "data.csv"
4 df = pd.read_csv(file_path)
5
6 print("===== ПЕРШІ 5 РЯДКІВ =====")
7 print(df.head())
8
9 # === 2. Аналіз структури ===
10 print("\n===== ІНФОРМАЦІЯ ПРО ДАНІ =====")
11 df.info()
12
13 print("\n===== ТИПИ ДАНИХ =====")
14 print(df.dtypes)
15
16 # === 3. Перевірка пропущених значень ===
17 print("\n===== ПРОПУЩЕНІ ЗНАЧЕННЯ =====")
18 missing_values = df.isnull().sum()
19 print(missing_values)
20
21 print("\n===== ВІДСОТОК ПРОПУЩЕНИХ ЗНАЧЕНЬ =====")
22 print((df.isnull().sum() / len(df)) * 100)
23
24 # === 4. Перевірка дублікатів ===
25 print("\n===== КІЛЬКІСТЬ ДУБЛІКАТІВ =====")
26 print(df.duplicated().sum())
27
28 # === 5. Описова статистика ===
29 print("\n===== ОПИСОВА СТАТИСТИКА =====")
30 print(df.describe(include='all'))
31
32 # === 6. Загальний висновок ===
33 print("\n===== ВИСНОВОК =====")
34
35 if missing_values.sum() == 0:
36     print("Пропущених значень немає.")
37 else:
38     print("Є пропущені значення – рекомендується обробка.")
39
40 if df.duplicated().sum() == 0:
41     print("Дублікатів не виявлено.")
42 else:
43     print("Є дублікати – рекомендується видалення.")
```

===== ПЕРШІ 5 РЯДКІВ =====

	Name	Age	City
0	yura	20	Lviv
1	antyk	19	wroclaw
2	komuto herovato	22	Kyoto
3	mina	20	munich

===== ІНФОРМАЦІЯ ПРО ДАНІ =====

```
<class 'pandas.DataFrame'>  
RangeIndex: 4 entries, 0 to 3  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0   Name    4 non-null        str  
1   Age     4 non-null        int64  
2   City    4 non-null        str  
dtypes: int64(1), str(2)  
memory usage: 228.0 bytes
```

===== ТИПИ ДАНИХ =====

```
Name      str  
Age       int64  
City      str  
dtype: object
```

===== ПРОПУЩЕНІ ЗНАЧЕННЯ =====

```
Name      0  
Age       0  
City      0  
dtype: int64
```

===== ВІДСОТОК ПРОПУЩЕНИХ ЗНАЧЕНЬ =====

```
Name      0.0  
Age       0.0  
City      0.0  
dtype: float64
```

===== КІЛЬКІСТЬ ДУБЛІКАТІВ =====

0

===== ОПИСОВА СТАТИСТИКА =====

	Name	Age	City
count	4	4.000000	4
unique	4	NaN	4
top	yura	NaN	Lviv
freq	1	NaN	1
mean	NaN	20.250000	NaN
std	NaN	1.258306	NaN
min	NaN	19.000000	NaN
25%	NaN	19.750000	NaN
50%	NaN	20.000000	NaN
75%	NaN	20.500000	NaN
max	NaN	22.000000	NaN

===== ВИСНОВОК =====

Пропущених значень немає.
Дублікатів не виявлено.

===== ВІДСОТОК ПРОПУЩЕНИХ ЗНАЧЕНЬ =====

```
Name      0.0  
Age       0.0  
City      0.0  
dtype: float64
```

===== КІЛЬКІСТЬ ДУБЛІКАТІВ =====

0

===== ОПИСОВА СТАТИСТИКА =====

	Name	Age	City
count	4	4.000000	4
unique	4	NaN	4
top	yura	NaN	Lviv
freq	1	NaN	1
mean	NaN	20.250000	NaN
std	NaN	1.258306	NaN
min	NaN	19.000000	NaN
25%	NaN	19.750000	NaN
50%	NaN	20.000000	NaN
75%	NaN	20.500000	NaN
max	NaN	22.000000	NaN

===== ВИСНОВОК =====

Пропущених значень немає.
Дублікатів не виявлено.

```
Age      0.0  
City     0.0
```

```
dtype: float64
```

===== КІЛЬКІСТЬ ДУБЛІКАТІВ =====

0

===== ОПИСОВА СТАТИСТИКА =====

	Name	Age	City
count	4	4.000000	4
unique	4	NaN	4
top	yura	NaN	Lviv
freq	1	NaN	1
mean	NaN	20.250000	NaN
std	NaN	1.258306	NaN
min	NaN	19.000000	NaN
25%	NaN	19.750000	NaN
50%	NaN	20.000000	NaN
75%	NaN	20.500000	NaN
max	NaN	22.000000	NaN

===== ВИСНОВОК =====

Пропущених значень немає.
Дублікатів не виявлено.

Фільтрація вакансій за умовами

Відібрати вакансії у певній сфері (наприклад, Cloud Computing).

Знайти вакансії з рівнем Senior.

Відібрати вакансії типу Full-Time у конкретному місті

```
etpy > ...
1  import pandas as pd
2
3  df = pd.read_csv("jobs.csv")
4  df.columns = df.columns.str.strip()
5
6  print("===== НАЗВИ СТОВПЦІВ =====")
7  print(df.columns)
8
9  if all(col in df.columns for col in ["Category", "Level", "Employment Type", "Location"]):
10
11     cloud_jobs = df[df["Category"] == "Cloud Computing"]
12     senior_jobs = df[df["Level"] == "Senior"]
13
14     city_name = "Kyiv"
15
16     fulltime_city_jobs = df[
17         (df["Employment Type"] == "Full-Time") &
18         (df["Location"] == city_name)
19     ]
20
21     print("\n===== Cloud Computing =====")
22     print(cloud_jobs)
23
24     print("\n===== Senior =====")
25     print(senior_jobs)
26
27     print(f"\n===== Full-Time у місті {city_name} =====")
28     print(fulltime_city_jobs)
29
30 else:
31     print("\nX Назви стовпців не співпадають. Подивіться список вище.")
```

```
===== НАЗВИ СТОВПЦІВ =====
Index(['Name', 'Age', 'City'], dtype='str')
```

Сортування даних за зарплатою

Відсортувати вакансії за рівнем оплати (Salary Range).

Вивести 5 вакансій з найвищою зарплатою.

Визначити, які посади є найбільш високооплачуваними.

Примітка: Salary Range подано у текстовому форматі.


```

1 import pandas as pd
2 import re
3 |
4 df = pd.read_csv("jobs.csv", sep=None, engine='python', encoding='utf-8-sig')
5
6 print("--- ПЕРЕВІРКА СТРУКТУРИ ---")
7 print(df.head(2))
8 print("-----")
9
10
11 SALARY_COL_INDEX = 1
12
13 target_col = df.columns[SALARY_COL_INDEX]
14 print(f"🚀 Працюємо зі стовпцем за індексом {SALARY_COL_INDEX}: '{target_col}'")
15
16 def extract_salary_range(salary_text):
17     text = str(salary_text).replace(',', '').replace(' ', '')
18     numbers = re.findall(r'\d+', text)
19     if len(numbers) >= 2:
20         return (int(numbers[0]) + int(numbers[1])) / 2
21     elif len(numbers) == 1:
22         return int(numbers[0])
23     return None
24
25 df["Average Salary"] = df[target_col].apply(extract_salary_range)
26
27 df_clean = df.dropna(subset=["Average Salary"])
28 df_sorted = df_clean.sort_values(by="Average Salary", ascending=False)
29
30 print("\n===== РЕЗУЛЬТАТ =====")
31 print(df_sorted.iloc[:5, [0, SALARY_COL_INDEX, -1]])

```

```

--- ПЕРЕВІРКА СТРУКТУРИ ---
      Category'  замість      Industry
Employment    Type'  замість  Employment_Type
Location'     замість      City           NaN
-----
🚀 Працюємо зі стовпцем за індексом 1: 'замість'

===== РЕЗУЛЬТАТ =====
Empty DataFrame
Columns: [Category', замість, Average Salary]
Index: []

```

Висновок:

Під час виконання цієї лабораторної роботи я опанував інструментарій бібліотеки **Pandas**. Зокрема, набув практичних навичок обробки та аналізу табличних даних: освоїв методи фільтрації, сортування та групування, а також застосування статистичних функцій для проведення аналітичних досліджень